Vyavahara — SRS & System Design Reference

Nov 3, 2026 · @riyaz nadaf

Vyavahara (ವ್ಯವಹಾರ) — ನಿಮ್ಮ ಪೂರ್ತಿ ವ್ಯವಹಾರ, ಒಂದೇ ಕಡೆ (all your business in one place). This is the single source of truth for building Vyavahara: an offline-first Android and web app, with a voice assistant, for village merchants to manage customer credit, supplier dues, stock and daily transactions. Version 1.0, owner: architecture lead.

How to use this document with AI coding tools

The AI follows this document literally, so every rule here is written as an instruction, and every item has a stable ID the AI can reference.

1. Work one build step at a time (section 11). Give the AI that step's prompt plus the sections it lists under "Read first".
2. When this document and the AI disagree, this document wins. If something is missing or ambiguous, the AI must ask, not guess, and the answer is added here.
3. Every pull request names the IDs it implements (e.g. `Implements FR-KH-03, VA-07`).
4. Words in **MUST**, **MUST NOT**, **SHOULD** and **MAY** carry their usual RFC 2119 meaning.
5. Code, table and column names in `code font` are exact. Do not rename them.

ID conventions

Prefix	Meaning	Example
FR-<module>-nn	Functional requirement	FR-KH-03 (khata)
VA-nn	Voice assistant rule	VA-07
NFR-nn	Non-functional requirement	NFR-01
SEC-nn	Security rule	SEC-04
SCR-nn	Screen	SCR-05
API-nn	API endpoint	API-02
M<n>	Build step (milestone)	M3

Prefix	Meaning	Example
C-nn	Client requirement in the client requirements doc (traceability)	C-FR-11

Module codes: AU auth & setup · CU customers · KH khata (credit/payment) · SU suppliers · ST stock · BL billing · DY daily summary & reports · NT notifications · SY sync · VA voice.

Contents

1. Introduction
2. Overall description
3. Architecture
4. Data model
5. Offline sync design and API contracts
6. Voice assistant design
7. Functional requirements (SRS)
8. Wireframes and screen specs
9. Non-functional requirements
10. Security, errors and edge cases
11. Build steps M0–M7
12. Testing strategy and traceability

1. Introduction

Purpose. This document specifies what Vyavahara must do (SRS) and how it is built (system design), in enough detail that a developer or AI tool can implement any build step without further explanation.

Scope of version 1. Merchant onboarding, customers and credit khata, supplier dues, stock, quick billing, daily and monthly summaries, reminders to customers by SMS or WhatsApp, a voice assistant in Kannada, Hindi and English, and offline-first sync between Android, web and cloud. Out of scope: customer login, online ordering, lending or interest, full accounting, GST billing (planned for later behind a feature flag).

Definitions

Term	Meaning
Merchant / owner	The shop owner who signs up; full rights
Helper	Staff or family member added by the owner; limited rights
Customer	A person who buys on credit; never logs in
Supplier	A wholesaler the shop buys from
Khata	The ledger of credit (customer owes) and payments for one customer
Credit entry	Customer takes goods now, pays later; increases the customer's balance
Payment entry	Customer pays; decreases the balance
Adjustment	A correction entry; signed (+ or −); always has a reason
Balance	Opening balance + credits − payments + adjustments. Positive = customer owes the shop
Supplier due	Purchases on credit − payments to the supplier
Paise	Money is stored as integer paise. ₹1 = 100 paise
Outbox	Local table of changes waiting to sync
Intent	What a voice command asks for, e.g. `add_credit`

References: client requirements doc (Voice Khata, 3 Nov 2026) · product blueprint (sections 1–17) · the build prompt for Claude.

2. Overall description

Vyavahara is a self-service platform: any merchant from any taluk or district signs up alone, works fully offline on a low-end phone, and syncs to the cloud when a network is available.

User classes and permissions

Action	Owner	Helper	Customer	Platform admin
Sign up shop, manage helpers, settings	Yes	No	—	No
Add customer / supplier / product	Yes	Yes	—	No
Add credit or payment entry	Yes	Yes; above `shops.helper_pin_limit_paise` needs owner PIN	—	No
Edit or delete any money entry (via adjustment)	Yes, reason required	No	—	No
Undo own last entry within 10 minutes	Yes	Yes	—	No
View balances, history, stock	Yes	Yes	—	No
View profit, cash-in-hand, reports, audit log	Yes	No	—	No
Send reminders	Yes	No	Receives only	No
Export khata (PDF/CSV)	Yes	No	—	No
See usage metrics (no names, no amounts per customer)	—	—	—	Yes

Operating environment

- Android 9+ (API 28+), 2–3 GB RAM, 16 GB storage, screens from 5 inches; 2G/3G/4G with long offline periods.
- Web: latest Chrome, Edge, Firefox on desktop or laptop; also Chrome on Android.
- Cloud: Supabase (Free plan at launch), region closest to India.

Design constraints

- Free tools and free tiers only (see the blueprint's section 16). Any paid service needs approval.
- No SMS OTP login (costs per message). Login = Google Sign-In or email OTP, then device PIN.
- Reminders are sent from the merchant's own phone (SMS intent or WhatsApp link); no paid messaging API in version 1.
- UPI: generate payment links and QR locally; the shop keeps its existing UPI soundbox.
- Every screen, message and spoken reply available in Kannada (`kn`), Hindi (`hi`) and English (`en`).

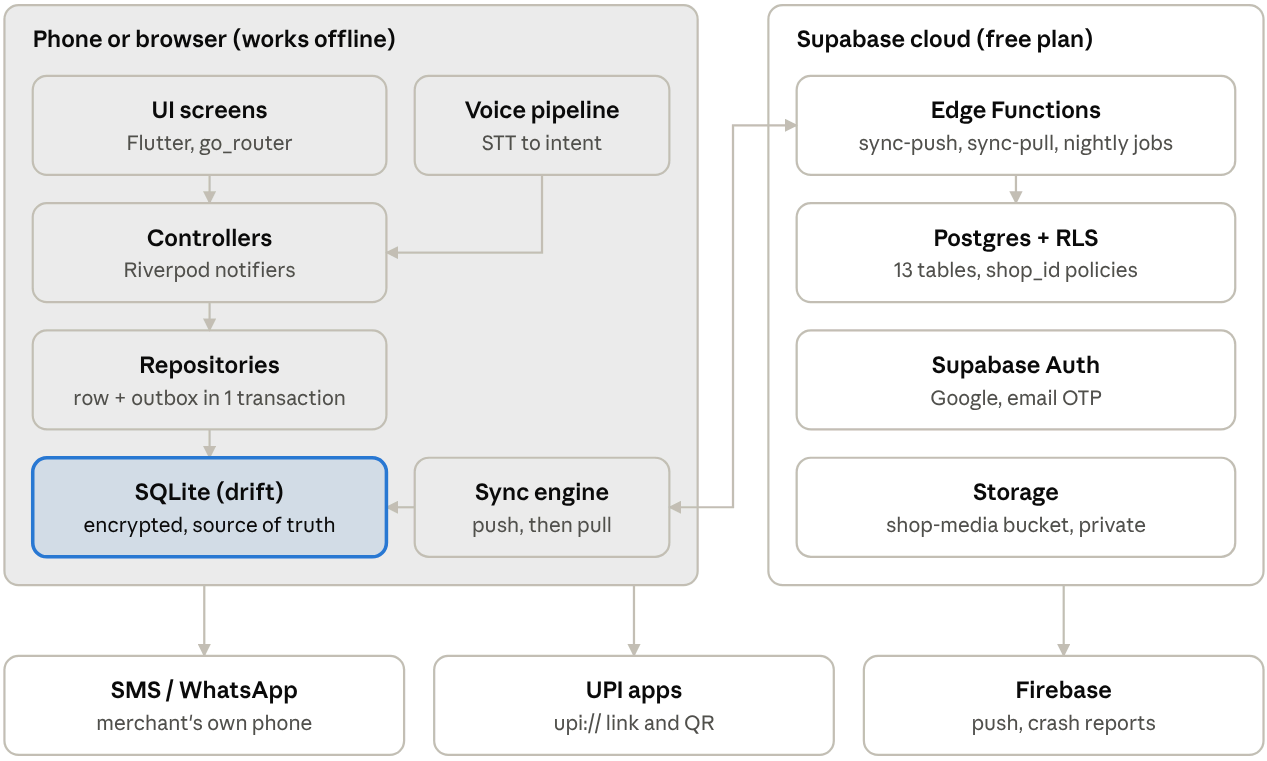
Assumptions and dependencies

- Each shop has at least one Android phone with WhatsApp.
- Internet is available at least once a day, even if weak.
- Android's on-device speech recognizer supports Kannada and Hindi on most target phones; where it doesn't, the app falls back to tap entry.
- Free plan limits: 500 MB database, 1 GB file storage, 50,000 monthly active users; the project pauses after a week with no activity.

3. Architecture

The device is the source of truth for daily work: every read and write hits the local SQLite database first, and a background sync engine exchanges changes with Supabase. The cloud adds backup, multi-device access, the web app and scheduled jobs; nothing at the counter waits for it.

Every write lands on the phone first; the cloud only syncs



system architecture · device, cloud, external services

The sync engine is the only component that talks to the cloud; UI, controllers and repositories never call the network directly.

Fixed tech stack (MUST NOT substitute without an approved ADR)

Layer	Choice	Version rule
App framework	Flutter + Dart, one codebase for Android and web	Latest stable
State management	`flutter_riverpod` (code generation with `riverpod_generator`)	Latest stable
Navigation	`go_router`	Latest stable
Local database	`drift` (SQLite; WASM on web) + `sqlcipher_flutter_libs` on Android	Latest stable
Cloud	Supabase: Postgres 15+, Auth, Storage, Edge Functions (TypeScript/Deno)	Free plan

Layer	Choice	Version rule
Auth	`supabase_flutter` with Google Sign-In and email OTP; local PIN via `flutter_secure_storage`	—
Voice	`speech_to_text` (on-device Android recognizer; Web Speech API on web), `flutter_tts`	—
Localisation	`flutter_localizations` + ARB files `app_kn.arb`, `app_hi.arb`, `app_en.arb`	—
Messaging	`url_launcher` for `sms:` and `https://wa.me/` links	—
Push / crashes	Firebase Cloud Messaging, Firebase Crashlytics	Free
PDF / CSV export	`pdf` + `printing`, `csv` packages	—
QR	`qr_flutter`	—
Tests	`flutter_test`, `mocktail`, `integration_test`; pgTAP or SQL tests for RLS	—
CI	GitHub Actions: analyze, test, build APK + web on every pull request	—

App layers (inside `/app/lib`)

Layer	Folder	Responsibility	Rule
UI	`features/<f>/ui/`	Screens and widgets only	No database or network calls
Controllers	`features/<f>/application/`	Riverpod notifiers; validation; calls repositories	No widget code
Domain	`features/<f>/domain/`	Plain Dart models, enums, value objects (`Money`, `Qty`)	No Flutter imports
Data	`features/<f>/data/`	Repositories over drift DAOs; writes outbox rows	Only layer touching the database

Layer	Folder	Responsibility	Rule
Core	`core/`	DB, sync engine, auth, l10n, theme, logging, `Money` formatting	Shared
Voice	`voice/`	STT adapter, normaliser, number parser, grammar, resolver, reply builder	Pure Dart except the STT/TTS adapters

Repository layout

```
vyavahara/
  app/                          Flutter app (Android + web)
    lib/
      core/{db,sync,auth,l10n,theme,money,logging}/

features/{onboarding,customers,khata,suppliers,stock,billing,daily,reminders,
settings}/
        {ui,application,domain,data}/
      voice/{stt,normalise,numbers,grammar,resolve,reply}/
      main.dart, router.dart
    l10n/app_kn.arb, app_hi.arb, app_en.arb
    test/  integration_test/
  supabase/
    migrations/  0001_init.sql, 0002_rls.sql, ...
    functions/   sync-push/, sync-pull/, nightly-jobs/
    seed.sql     config.toml
  docs/
    srs.md (this document), adr/, voice-grammar.md, data-model.md
  .github/workflows/ci.yml  ISSUE_TEMPLATE/  PULL_REQUEST_TEMPLATE.md
  README.md  CONTRIBUTING.md  .env.example
```

Environments: `dev` (Supabase project `vyavahara-dev`, debug builds) and `prod` (`vyavahara-prod`, signed release). Keys live in `.env` locally and GitHub Actions secrets in CI; `.env` MUST NOT be committed.

4. Data model

Three ledgers carry all money and stock: `ledger_entries` (customers), `supplier_entries` (suppliers) and `stock_movements` (items). All three are append-only with signed amounts, so every balance is a plain `SUM` and two devices can never overwrite each other's entries.

Global rules (apply to every table)

1. Primary key `id UUID`, generated on the device (UUID v4). The server MUST NOT generate ids for synced rows.
2. Every synced table has `shop_id`, `created_at`, `updated_at`, `deleted_at` (soft delete), `created_by`, `device_id`, `server_seq`.
3. `server_seq BIGINT` is set by a database trigger from one global sequence on every insert and update. Devices pull changes with `server_seq > last_seen`.
4. Money is `BIGINT` paise. Quantities are `NUMERIC(12,3)`. Never use floating point for either.
5. Rows are never hard-deleted by the app. A delete sets `deleted_at`.
6. Ledger rows (`ledger_entries`, `supplier_entries`, `stock_movements`) are immutable after insert except for setting `deleted_at` (undo). Corrections are new `adjustment` rows.
7. Timestamps are `TIMESTAMPTZ` in UTC; the app displays local time (Asia/Kolkata by default, per-shop setting).

Sign conventions

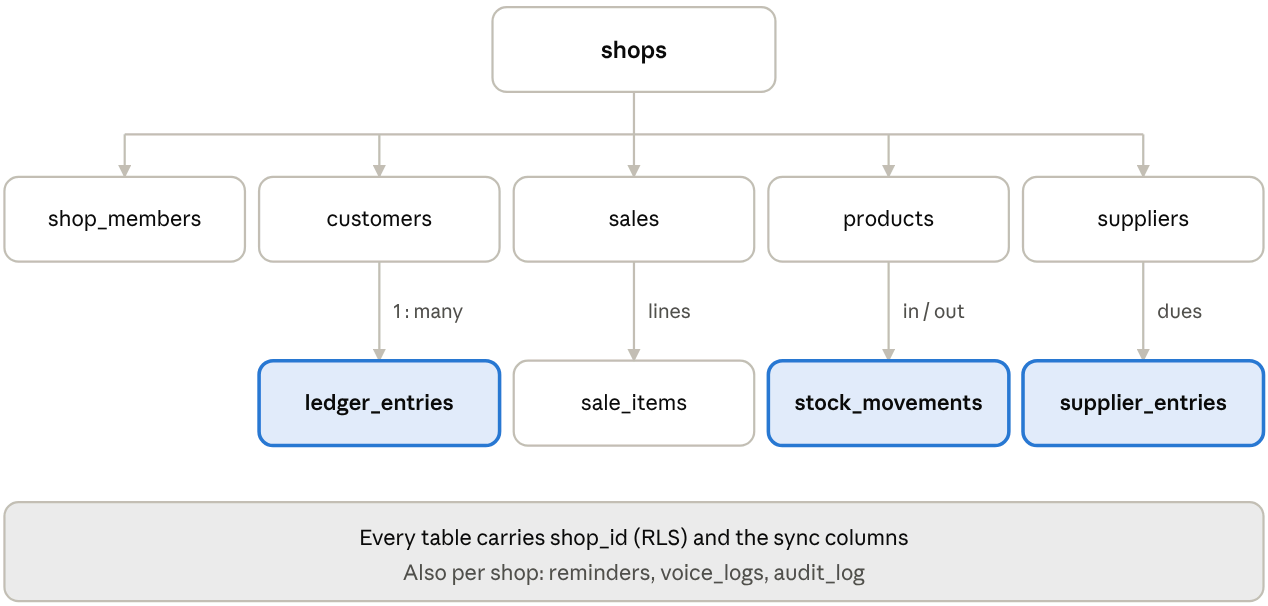
Ledger	Positive amount means	Negative amount means
`ledger_entries.amount_paise`	Customer owes more (`credit`, + `adjustment`)	Customer paid (`payment`, − `adjustment`)
`supplier_entries.amount_paise`	Shop owes supplier more (`purchase`)	Shop paid supplier (`payment`)
`stock_movements.qty_change`	Stock in (`purchase`, `return_in`, `opening`, + `adjustment`)	Stock out (`sale`, `damage`, − `adjustment`)

Tables

Table	Purpose	Key columns
`shops`	One per merchant business	name, owner_name, phone, village, taluk, district, state, upi_vpa, default_lang, timezone, helper_pin_limit_paise, plan, trial_ends_at, gst_enabled
`shop_members`	Links Supabase auth users to shops with a role	shop_id, user_id, role (`owner` / `helper`), display_name, pin_hash, is_active
`customers`	Credit customers	full_name, name_latin, nickname, phonetic_key, phone, village, photo_path, lang, credit_limit_paise, opening_balance_paise, reminder_opt_in
`suppliers`	Wholesalers	name, name_latin, phone, town, opening_due_paise
`products`	Items sold	name_local, name_latin, aliases, unit, sell_price_paise, cost_price_paise, reorder_level, barcode, category
`sales`	One bill	customer_id (nullable), sale_time, total_paise, cash_paise, upi_paise, credit_paise, source
`sale_items`	Lines of a bill	sale_id, product_id (nullable), item_text, qty, unit, unit_price_paise, line_total_paise
`ledger_entries`	Customer credits, payments, adjustments	customer_id, entry_type, amount_paise (signed), payment_mode, sale_id, note, entry_time, reason, source
`supplier_entries`	Supplier purchases and payments	supplier_id, entry_type, amount_paise (signed), payment_mode, bill_photo_path, note, entry_time

Table	Purpose	Key columns
`stock_movements`	Every stock change	product_id, movement_type, qty_change (signed), unit_cost_paise, sale_id, supplier_entry_id, expiry_date, movement_time
`reminders`	Reminders the merchant sent	customer_id, channel, balance_paise, message_text, lang, sent_at
`voice_logs`	Voice commands (text only)	transcript, lang, intent, entities (JSON), confidence, engine, outcome
`audit_log`	Every correction, delete, settings change	table_name, record_id, action, old_values, new_values, reason

Three append-only ledgers hold all money and stock



entity relationships · 13 synced tables

Cross-links not drawn: a bill's credit part creates a `ledger_entries` row with `sale_id`; each `sale_items` line creates a `stock_movements` row with `sale_id`; a purchase's stock lines link through `supplier_entry_id`.

Device-only tables (drift, not in Postgres): `outbox` (pending changes) and `sync_state` (last pulled `server_seq`, last sync time).

Full Postgres DDL — `supabase/migrations/0001_init.sql`

```
CREATE EXTENSION IF NOT EXISTS pgcrypto;
CREATE EXTENSION IF NOT EXISTS pg_trgm;

CREATE SEQUENCE sync_seq;

CREATE OR REPLACE FUNCTION set_sync_fields() RETURNS trigger AS $$
BEGIN
  NEW.server_seq := nextval('sync_seq');
  IF TG_OP = 'UPDATE' THEN NEW.updated_at := GREATEST(NEW.updated_at,
OLD.updated_at); END IF;
  RETURN NEW;
END $$ LANGUAGE plpgsql;

-- Enums
CREATE TYPE member_role   AS ENUM ('owner','helper');
CREATE TYPE lang_code     AS ENUM ('kn','hi','en');
CREATE TYPE ledger_type   AS ENUM ('credit','payment','adjustment');
CREATE TYPE supplier_type AS ENUM ('purchase','payment','adjustment');
CREATE TYPE pay_mode      AS ENUM ('cash','upi','other');
CREATE TYPE unit_code     AS ENUM
('kg','g','l','ml','pcs','pkt','dozen','box');
CREATE TYPE move_type     AS ENUM
('opening','purchase','sale','return_in','damage','adjustment');
CREATE TYPE entry_source  AS ENUM ('manual','voice','import');
CREATE TYPE channel_code  AS ENUM ('sms','whatsapp');

CREATE TABLE shops (
  id UUID PRIMARY KEY,
  name TEXT NOT NULL CHECK (length(name) BETWEEN 2 AND 120),
  owner_name TEXT NOT NULL,
  phone TEXT,
  village TEXT, taluk TEXT, district TEXT, state TEXT,
  upi_vpa TEXT,
  default_lang lang_code NOT NULL DEFAULT 'kn',
  timezone TEXT NOT NULL DEFAULT 'Asia/Kolkata',
  helper_pin_limit_paise BIGINT NOT NULL DEFAULT 50000,      -- Rs 500
  plan TEXT NOT NULL DEFAULT 'free_intro',
  trial_ends_at DATE,
  gst_enabled BOOLEAN NOT NULL DEFAULT FALSE,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT now(),
```

```
  deleted_at TIMESTAMPTZ,
  created_by UUID, device_id TEXT,
  server_seq BIGINT
);

CREATE TABLE shop_members (
  id UUID PRIMARY KEY,
  shop_id UUID NOT NULL REFERENCES shops(id),
  user_id UUID REFERENCES auth.users(id),          -- NULL until helper
accepts invite
  invite_email TEXT,
  role member_role NOT NULL,
  display_name TEXT NOT NULL,
  pin_hash TEXT,                                   -- argon2id; verified on
device
  is_active BOOLEAN NOT NULL DEFAULT TRUE,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  deleted_at TIMESTAMPTZ,
  created_by UUID, device_id TEXT, server_seq BIGINT,
  UNIQUE (shop_id, user_id)
);

CREATE TABLE customers (
  id UUID PRIMARY KEY,
  shop_id UUID NOT NULL REFERENCES shops(id),
  full_name TEXT NOT NULL CHECK (length(full_name) BETWEEN 1 AND 100),
  name_latin TEXT NOT NULL,
  nickname TEXT,
  phonetic_key TEXT,
  phone TEXT CHECK (phone IS NULL OR phone ~ '^[6-9][0-9]{9}$'),
  village TEXT,
  photo_path TEXT,
  lang lang_code NOT NULL DEFAULT 'kn',
  credit_limit_paise BIGINT CHECK (credit_limit_paise IS NULL OR
credit_limit_paise >= 0),
  opening_balance_paise BIGINT NOT NULL DEFAULT 0,
  reminder_opt_in BOOLEAN NOT NULL DEFAULT FALSE,
  notes TEXT,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  deleted_at TIMESTAMPTZ,
  created_by UUID, device_id TEXT, server_seq BIGINT
```

```
);
CREATE INDEX ix_customers_shop ON customers(shop_id) WHERE deleted_at IS
NULL;
CREATE INDEX ix_customers_trgm ON customers USING gin (name_latin
gin_trgm_ops);

CREATE TABLE suppliers (
  id UUID PRIMARY KEY,
  shop_id UUID NOT NULL REFERENCES shops(id),
  name TEXT NOT NULL, name_latin TEXT NOT NULL,
  phone TEXT, town TEXT,
  opening_due_paise BIGINT NOT NULL DEFAULT 0,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  deleted_at TIMESTAMPTZ,
  created_by UUID, device_id TEXT, server_seq BIGINT
);

CREATE TABLE products (
  id UUID PRIMARY KEY,
  shop_id UUID NOT NULL REFERENCES shops(id),
  name_local TEXT NOT NULL,
  name_latin TEXT NOT NULL,
  aliases TEXT[] NOT NULL DEFAULT '{}',            -- e.g. {akki,chawal,rice}
  category TEXT,
  unit unit_code NOT NULL,
  sell_price_paise BIGINT NOT NULL CHECK (sell_price_paise >= 0),
  cost_price_paise BIGINT CHECK (cost_price_paise IS NULL OR cost_price_paise
>= 0),
  reorder_level NUMERIC(12,3) NOT NULL DEFAULT 0,
  barcode TEXT,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  deleted_at TIMESTAMPTZ,
  created_by UUID, device_id TEXT, server_seq BIGINT
);

CREATE TABLE sales (
  id UUID PRIMARY KEY,
  shop_id UUID NOT NULL REFERENCES shops(id),
  customer_id UUID REFERENCES customers(id),
  sale_time TIMESTAMPTZ NOT NULL,
  total_paise BIGINT NOT NULL CHECK (total_paise >= 0),
```

```
  cash_paise BIGINT NOT NULL DEFAULT 0 CHECK (cash_paise >= 0),
  upi_paise BIGINT NOT NULL DEFAULT 0 CHECK (upi_paise >= 0),
  credit_paise BIGINT NOT NULL DEFAULT 0 CHECK (credit_paise >= 0),
  CHECK (cash_paise + upi_paise + credit_paise = total_paise),
  CHECK (credit_paise = 0 OR customer_id IS NOT NULL),
  source entry_source NOT NULL DEFAULT 'manual',
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  deleted_at TIMESTAMPTZ,
  created_by UUID NOT NULL, device_id TEXT, server_seq BIGINT
);
CREATE INDEX ix_sales_shop_time ON sales(shop_id, sale_time);

CREATE TABLE sale_items (
  id UUID PRIMARY KEY,
  shop_id UUID NOT NULL REFERENCES shops(id),
  sale_id UUID NOT NULL REFERENCES sales(id),
  product_id UUID REFERENCES products(id),
  item_text TEXT,
  qty NUMERIC(12,3) NOT NULL CHECK (qty > 0),
  unit unit_code NOT NULL,
  unit_price_paise BIGINT NOT NULL CHECK (unit_price_paise >= 0),
  line_total_paise BIGINT NOT NULL CHECK (line_total_paise >= 0),
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  deleted_at TIMESTAMPTZ,
  created_by UUID, device_id TEXT, server_seq BIGINT
);

CREATE TABLE ledger_entries (
  id UUID PRIMARY KEY,
  shop_id UUID NOT NULL REFERENCES shops(id),
  customer_id UUID NOT NULL REFERENCES customers(id),
  entry_type ledger_type NOT NULL,
  amount_paise BIGINT NOT NULL,
  CHECK ((entry_type = 'credit' AND amount_paise > 0)
      OR (entry_type = 'payment' AND amount_paise < 0)
      OR (entry_type = 'adjustment' AND amount_paise <> 0)),
  payment_mode pay_mode,
  CHECK (entry_type <> 'payment' OR payment_mode IS NOT NULL),
  sale_id UUID REFERENCES sales(id),
  note TEXT,
  reason TEXT,
```

```
  CHECK (entry_type <> 'adjustment' OR length(reason) > 0),
  entry_time TIMESTAMPTZ NOT NULL,
  due_date DATE,
  source entry_source NOT NULL DEFAULT 'manual',
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  deleted_at TIMESTAMPTZ,
  created_by UUID NOT NULL, device_id TEXT, server_seq BIGINT
);
CREATE INDEX ix_ledger_customer ON ledger_entries(customer_id, entry_time);
CREATE INDEX ix_ledger_shop_time ON ledger_entries(shop_id, entry_time);

CREATE TABLE supplier_entries (
  id UUID PRIMARY KEY,
  shop_id UUID NOT NULL REFERENCES shops(id),
  supplier_id UUID NOT NULL REFERENCES suppliers(id),
  entry_type supplier_type NOT NULL,
  amount_paise BIGINT NOT NULL,
  CHECK ((entry_type = 'purchase' AND amount_paise > 0)
      OR (entry_type = 'payment' AND amount_paise < 0)
      OR (entry_type = 'adjustment' AND amount_paise <> 0)),
  payment_mode pay_mode,
  bill_photo_path TEXT,
  note TEXT, reason TEXT,
  entry_time TIMESTAMPTZ NOT NULL,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  deleted_at TIMESTAMPTZ,
  created_by UUID NOT NULL, device_id TEXT, server_seq BIGINT
);

CREATE TABLE stock_movements (
  id UUID PRIMARY KEY,
  shop_id UUID NOT NULL REFERENCES shops(id),
  product_id UUID NOT NULL REFERENCES products(id),
  movement_type move_type NOT NULL,
  qty_change NUMERIC(12,3) NOT NULL CHECK (qty_change <> 0),
  unit_cost_paise BIGINT,
  sale_id UUID REFERENCES sales(id),
  supplier_entry_id UUID REFERENCES supplier_entries(id),
  expiry_date DATE,
  reason TEXT,
  movement_time TIMESTAMPTZ NOT NULL,
```

```
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  deleted_at TIMESTAMPTZ,
  created_by UUID NOT NULL, device_id TEXT, server_seq BIGINT
);
CREATE INDEX ix_stock_product ON stock_movements(product_id, movement_time);

CREATE TABLE reminders (
  id UUID PRIMARY KEY,
  shop_id UUID NOT NULL REFERENCES shops(id),
  customer_id UUID NOT NULL REFERENCES customers(id),
  channel channel_code NOT NULL,
  balance_paise BIGINT NOT NULL,
  message_text TEXT NOT NULL,
  lang lang_code NOT NULL,
  sent_at TIMESTAMPTZ NOT NULL,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  deleted_at TIMESTAMPTZ,
  created_by UUID NOT NULL, device_id TEXT, server_seq BIGINT
);

CREATE TABLE voice_logs (
  id UUID PRIMARY KEY,
  shop_id UUID NOT NULL REFERENCES shops(id),
  transcript TEXT NOT NULL,
  lang TEXT,
  intent TEXT,
  entities JSONB,
  confidence NUMERIC(4,3),
  engine TEXT,                                    -- android_ondevice,
web_speech, bhashini
  outcome TEXT CHECK (outcome IN
('confirmed','corrected','cancelled','failed','answered')),
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  deleted_at TIMESTAMPTZ,
  created_by UUID, device_id TEXT, server_seq BIGINT
);

CREATE TABLE audit_log (
  id UUID PRIMARY KEY,
  shop_id UUID NOT NULL REFERENCES shops(id),
```

```
  table_name TEXT NOT NULL,
  record_id UUID NOT NULL,
  action TEXT NOT NULL CHECK (action IN
('adjust','undo','update','settings','conflict')),
  old_values JSONB, new_values JSONB,
  reason TEXT,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  deleted_at TIMESTAMPTZ,
  created_by UUID, device_id TEXT, server_seq BIGINT
);

-- Attach the sync trigger to every synced table
DO $$
DECLARE t TEXT;
BEGIN
  FOREACH t IN ARRAY
ARRAY['shops','shop_members','customers','suppliers','products','sales',

'sale_items','ledger_entries','supplier_entries','stock_movements','reminders
','voice_logs','audit_log']
  LOOP
    EXECUTE format('CREATE TRIGGER trg_sync_%1$s BEFORE INSERT OR UPDATE ON
%1$s
                    FOR EACH ROW EXECUTE FUNCTION set_sync_fields()', t);
    IF t <> 'shops' THEN
      EXECUTE format('CREATE INDEX ix_%1$s_seq ON %1$s(shop_id, server_seq)',
t);
    END IF;
  END LOOP;
END $$;

-- Views used by reports and voice answers
CREATE VIEW v_customer_balance AS
SELECT c.shop_id, c.id AS customer_id, c.full_name, c.village, c.phone,
c.credit_limit_paise,
       c.opening_balance_paise + COALESCE(SUM(l.amount_paise), 0) AS
balance_paise,
       MAX(l.entry_time) FILTER (WHERE l.entry_type = 'credit')  AS
last_credit_at,
       MAX(l.entry_time) FILTER (WHERE l.entry_type = 'payment') AS
last_payment_at
FROM customers c
```

```
LEFT JOIN ledger_entries l ON l.customer_id = c.id AND l.deleted_at IS NULL
WHERE c.deleted_at IS NULL
GROUP BY c.id;

CREATE VIEW v_supplier_due AS
SELECT s.shop_id, s.id AS supplier_id, s.name,
       s.opening_due_paise + COALESCE(SUM(e.amount_paise), 0) AS due_paise,
       MAX(e.entry_time) FILTER (WHERE e.entry_type = 'payment') AS
last_payment_at
FROM suppliers s
LEFT JOIN supplier_entries e ON e.supplier_id = s.id AND e.deleted_at IS NULL
WHERE s.deleted_at IS NULL
GROUP BY s.id;

CREATE VIEW v_stock_on_hand AS
SELECT p.shop_id, p.id AS product_id, p.name_local, p.name_latin, p.unit,
p.reorder_level,
       COALESCE(SUM(m.qty_change), 0) AS qty_on_hand
FROM products p
LEFT JOIN stock_movements m ON m.product_id = p.id AND m.deleted_at IS NULL
WHERE p.deleted_at IS NULL
GROUP BY p.id;
```

The device (drift) MUST mirror these tables column for column, with `TEXT` for UUIDs and ISO-8601 text or integer milliseconds for timestamps, and the same three views written as drift queries. Any schema change is a new numbered migration in both places, in the same pull request.

Row Level Security — `supabase/migrations/0002_rls.sql`

```
CREATE OR REPLACE FUNCTION is_member(p_shop UUID) RETURNS BOOLEAN
LANGUAGE sql STABLE SECURITY DEFINER SET search_path = public AS $$
  SELECT EXISTS (SELECT 1 FROM shop_members
                 WHERE shop_id = p_shop AND user_id = auth.uid()
                   AND is_active AND deleted_at IS NULL);
$$;

CREATE OR REPLACE FUNCTION is_owner(p_shop UUID) RETURNS BOOLEAN
LANGUAGE sql STABLE SECURITY DEFINER SET search_path = public AS $$
  SELECT EXISTS (SELECT 1 FROM shop_members
                 WHERE shop_id = p_shop AND user_id = auth.uid()
                   AND role = 'owner' AND is_active AND deleted_at IS NULL);
$$;
```

```
-- Enable RLS on every table, then: members read; members insert; no hard
deletes.
-- Example for ledger_entries (repeat the pattern per table):
ALTER TABLE ledger_entries ENABLE ROW LEVEL SECURITY;

CREATE POLICY le_select ON ledger_entries FOR SELECT USING
(is_member(shop_id));

CREATE POLICY le_insert ON ledger_entries FOR INSERT WITH CHECK (
  is_member(shop_id) AND created_by = auth.uid()
  AND (entry_type <> 'adjustment' OR is_owner(shop_id))
);

-- Only change allowed after insert: soft delete (undo) by the creator within
10 minutes, or by the owner.
CREATE POLICY le_update ON ledger_entries FOR UPDATE USING
(is_member(shop_id)) WITH CHECK (
  is_owner(shop_id)
  OR (created_by = auth.uid() AND deleted_at IS NOT NULL
      AND deleted_at - created_at <= interval '10 minutes')
);
-- No DELETE policy = hard deletes are refused.

-- Owner-only tables/actions: shop settings, shop_members, audit_log read,
reports.
CREATE POLICY shops_update ON shops FOR UPDATE USING (is_owner(id));
CREATE POLICY members_write ON shop_members FOR ALL USING (is_owner(shop_id))
WITH CHECK (is_owner(shop_id));
CREATE POLICY audit_select ON audit_log FOR SELECT USING (is_owner(shop_id));
```

A trigger on `ledger_entries`, `supplier_entries` and `stock_movements` MUST reject any update that changes a column other than `deleted_at` and `updated_at` (enforces immutability).

5. Offline sync design and API contracts

Sync is push-then-pull over two Edge Functions, called with the user's own login token so Row Level Security applies to every row. Because ledgers are append-only with device-made UUIDs, the only real conflicts are on master data (customer, product, supplier, settings), and those resolve by last `updated_at` wins.

Device tables for sync (drift only)

```
CREATE TABLE outbox (
  seq INTEGER PRIMARY KEY AUTOINCREMENT,
  table_name TEXT NOT NULL,
  row_id TEXT NOT NULL,
  payload TEXT NOT NULL,          -- full row as JSON, as it is after the
change
  created_at INTEGER NOT NULL,    -- epoch ms
  attempts INTEGER NOT NULL DEFAULT 0,
  last_error TEXT,
  status TEXT NOT NULL DEFAULT 'pending'   -- pending | rejected
);
CREATE TABLE sync_state (
  table_name TEXT PRIMARY KEY,
  last_server_seq INTEGER NOT NULL DEFAULT 0,
  last_pulled_at INTEGER
);
```

Write rule (every repository method that changes data) — in ONE local transaction: write the row, then insert an `outbox` row with the full row JSON. If either fails, both roll back.

Sync algorithm (`core/sync/sync_engine.dart`)

1. Triggers: app start, network becomes available, every 5 minutes while the app is open, pull-to-refresh, and after each write (debounced 10 s).
2. Only one sync runs at a time (mutex). No sync without a valid session.
3. **Push:** take up to 100 `pending` outbox rows in `seq` order → call API-01. For each id in `applied`, delete its outbox row. For each in `rejected` with a permanent code, set `status = 'rejected'` and show it under Settings › Sync problems. On network or 5xx errors, increase `attempts` and back off (30 s, 1 min, 2 min … max 15 min). Repeat until the outbox is empty.
4. **Pull:** call API-02 with the `last_server_seq` per table. Upsert every returned row locally **without** writing to the outbox. Store the new cursors. Repeat while `has_more` is true.
5. Local upsert rule for master data: apply the incoming row only if its `updated_at` ≥ the local row's; if the local row is newer and still in the outbox, keep local (it will win on push).

6. Show sync status on Home: `synced` (green), `N waiting` (orange), `offline for X days` (red, after 3 days).
7. New device or reinstall: cursors start at 0, so the full shop downloads (bootstrap). Show a progress screen.

API-01 — `POST /functions/v1/sync-push`

```
// Request (Authorization: Bearer <user JWT>)
{
  "device_id": "a1b2c3",
  "shop_id": "6f0c…",
  "changes": [
    { "table": "ledger_entries", "row": { "id": "…", "customer_id": "…",
"entry_type": "credit",
      "amount_paise": 20000, "note": "rice, oil", "entry_time": "2026-11-
03T08:12:00Z",
      "source": "voice", "created_by": "…", "created_at": "…", "updated_at":
"…", "deleted_at": null } }
  ]
}
// Response 200
{
  "applied": ["<id>", "<id>"],
  "rejected": [ { "id": "<id>", "code": "RLS_DENIED", "message": "helper
cannot add adjustment" } ]
}
```

Server rules: max 100 changes per call; tables allowed = the 13 synced tables only; upsert `ON CONFLICT (id)`. For ledger tables an existing row may only change `deleted_at` / `updated_at`; for master tables update only if incoming `updated_at` ⩾ stored, else return it in `applied` but write an `audit_log` row with action `conflict`. Rejection codes: `RLS_DENIED`, `VALIDATION`, `IMMUTABLE`, `UNKNOWN_TABLE`, `FK_MISSING` (retry later: parent not yet synced — this one is NOT permanent).

API-02 — `POST /functions/v1/sync-pull`

```
// Request
{ "shop_id": "6f0c…", "cursors": { "customers": 1840, "ledger_entries": 1902,
"products": 0 }, "limit": 500 }
// Response 200
{
  "changes": { "customers": [ { "id": "…", "server_seq": 1851, "...": "..." }
], "ledger_entries": [] },
  "cursors": { "customers": 1851, "ledger_entries": 1902, "products": 0 },
  "has_more": false
}
```

Order of tables in push and pull (parents first): `shops, shop_members, customers, suppliers, products, sales, sale_items, ledger_entries, supplier_entries, stock_movements, reminders, voice_logs, audit_log`.

Other endpoints

ID	Endpoint	Purpose	Notes
API-03	`rpc/create_shop(p_shop JSON, p_owner JSON)`	Creates the shop and the owner's `shop_members` row in one transaction	Called once at onboarding when online; if offline, onboarding waits for network (only step that needs internet)
API-04	`rpc/invite_helper(p_shop UUID, p_email TEXT, p_name TEXT)`	Owner invites a helper by email	Creates a `shop_members` row with `user_id` NULL
API-05	`rpc/accept_invite(p_member UUID)`	Helper claims the invite after logging in	Sets `user_id = auth.uid()`
API-06	Storage bucket `shop-media`, path `{shop_id}/{table}/{id}.jpg`	Customer photos, bill photos	Images compressed to ⩽ 100 KB on device; uploaded on Wi-Fi or when the user taps "upload now"; RLS by `shop_id` folder

ID	Endpoint	Purpose	Notes
API-07	`functions/v1/nightly-jobs` (scheduled)	Usage counts per shop (no names), database size check, purge `voice_logs` older than 90 days	Runs once a day
API-08	Supabase Auth	Google Sign-In, email OTP, refresh tokens	Session kept 30 days; app also locked by local PIN

6. Voice assistant design

The assistant is a deterministic pipeline of pure Dart functions: speech to text, normalise, parse intent, resolve names, confirm, execute, reply. No language model sits in the money path, so it works offline and never invents a number.

Pipeline contracts (`app/lib/voice/`)

Step	File	Input → Output	Rule
1 Capture	`stt/stt_adapter.dart`	mic audio → `SttResult{text, alternatives[], locale, confidence}`	Interface `SttEngine` with `AndroidOnDeviceStt`, `WebSpeechStt`; later `BhashiniStt`. Push-to-talk, max 15 s, partial results shown live
2 Normalise	`normalise/normaliser.dart`	text → `NormText{tokens[], latin}`	Lower-case; transliterate Kannada and Devanagari to Latin (ISO-15919-like table in `voice-grammar.md`); strip punctuation; merge split numbers
3 Numbers	`numbers/number_parser.dart`	tokens → tokens with `NUM(value)`	Kannada, Hindi, English number words and digits; see rules below
4 Intent	`grammar/intent_parser.dart`	tokens → `ParsedCommand{intent, slots, confidence}`	Keyword + pattern rules per intent and language; highest score wins; confidence < 0.6 = not understood

Step	File	Input → Output	Rule
5 Resolve	`resolve/entity_resolver.dart`	slots → `ResolvedCommand` with ids	Customer/product/supplier matching (below)
6 Confirm	`ui/voice_confirm_sheet.dart`	`ResolvedCommand` → user yes/no	Required for every write intent (VA-04)
7 Execute	existing repositories	command → saved rows	Same code path as manual entry; `source = 'voice'`
8 Reply	`reply/reply_builder.dart`	result → localised text → TTS	Template strings from ARB files only

Intents (version 1)

Intent	Write?	Required slots	Optional slots	Example (kn / hi / en)
`get_balance`	No	customer	—	Ramesh eshtu baaki? / Ramesh ka kitna baaki? / How much does Ramesh owe?
`add_credit`	Yes	customer, amount	items[], date	Suresh ge 200 saala, akki enne / Suresh ke khaate mein 200 likho / Add 200 credit for Suresh
`add_payment`	Yes	customer, amount	mode (cash/upi), date	Lakshmi 500 kottidaare / Lakshmi ne 500 diye / Lakshmi paid 500
`list_overdue`	No	—	min_amount, days	Yaaru 1000 kinta jaasti kodbeku? / Kisne mahine se nahi diya? / Who owes more than 1000?

Intent	Write?	Required slots	Optional slots	Example (kn / hi / en)
`today_summary`	No	—	period (today/week/month)	Ivattu vyapaara eshtu? / Aaj kitni bikri? / Today's sales?
`stock_check`	No	product	—	Sakkare eshtu ide? / Cheeni kitni hai? / How much sugar is left?
`add_stock`	Yes	product, qty	unit, unit_cost	Ippattu kilo akki banthu / 20 kilo chawal aaya / Add 20 kg rice at 45
`low_stock`	No	—	—	Yaava saamaanu mugiyutide? / Kya khatam ho raha hai? / What is running low?
`supplier_due`	No	—	supplier	Wholesaler ge eshtu kodbeku? / Supplier ka kitna dena hai? / How much do I owe suppliers?
`send_reminder`	Yes (opens share)	customer or `all_overdue`	—	Ellarigu reminder kalisu / Sabko reminder bhejo / Send reminders
`undo_last`	Yes	—	—	Kone entry tegi / Pichhli entry hatao / Undo the last entry

Intent	Write?	Required slots	Optional slots	Example (kn / hi / en)
`confirm` / `cancel`	—	—	—	haudu, sari, howdu / haan, theek, sahi / yes, ok · illa, beda / nahi, mat / no, cancel

Keyword lexicon (seed; full list in `docs/voice-grammar.md`)

Concept	Kannada (Latin)	Hindi (Latin)	English
credit	saala, baaki bari, udhaari, ge kodi	udhaar, khaate mein likho, baaki likho	credit, udhaar, on account
payment	kottidaare, kottaru, jama, vasool	diye, de diya, jama, mila	paid, payment, gave
balance query	eshtu baaki, eshtu kodbeku	kitna baaki, kitna dena	how much, owe, balance
stock	eshtu ide, stock	kitna hai, maal	left, stock, how much
arrived	banthu, bandide	aaya, aa gaya	arrived, add stock, got
UPI	PhonePe, GPay, Paytm, UPI, online	same	same

Number parsing rules (`number_parser.dart`)

1. Digits pass through (`200`, `1,500`, `1500.50` → rupees; paise only if a decimal is spoken).
2. Kannada: ondu 1, eradu 2, mooru 3, naalku 4, aidu 5, aaru 6, elu 7, entu 8, ombattu 9, hattu 10, ippattu 20, moovattu 30, nalavattu 40, aivattu 50, nooru 100, innooru 200, munnooru 300, naanooru 400, ainooru 500, saavira 1000, laksha 100000. Compounds add: "ondu saavira aidu nooru" = 1500.

3. Hindi: ek … sau 100, hazaar 1000, lakh 100000; specials: dedh 1.5×, dhai 2.5×, saadhe (+0.5), sava (+0.25), paune (−0.25) applied to the next unit ("dhai sau" = 250, "saadhe teen sau" = 350).
4. English: one … thousand, lakh; "fifteen hundred" = 1500.
5. Ambiguous pairs (fifteen/fifty, nineteen/ninety): pick the transcript's top alternative, but always show digits on the confirm card and speak them back.
6. Quantity units: kg/kilo/kejee, gram, litre/leetar, packet/paakit, piece/peesu, dozen/darjan, box → `unit_code`.
7. Unit tests: ≥ 60 phrases per language in `test/voice/number_parser_test.dart`.

Entity resolution (`entity_resolver.dart`)

1. Candidates = active customers of this shop (or products/suppliers for those slots).
2. Score = max of: exact match on `name_latin` or `nickname` (1.0); `phonetic_key` equal (0.9); trigram similarity on `name_latin` (0–1); alias match for products (1.0).
3. One candidate ≥ 0.85 and the next < 0.70 → resolved.
4. Two or more ≥ 0.70 → ask: "Which Suresh — Lokapur or Bagalkot?" and show up to 3 cards with photo, village and balance. Owner taps or says the village or nickname.
5. None ≥ 0.70 → "I couldn't find Suresh. Add as a new customer?" (write intents) or "not found" (questions).
6. Remember the last choice for 30 days: if the owner picked customer X for spoken "Suresh", give X +0.1 next time.

Voice rules

ID	Rule
VA-01	The mic button is present on every main screen, bottom centre, ≥ 72 dp
VA-02	Recognition language = the app language, with the other two as alternatives where the engine allows
VA-03	Partial transcript shown live; final transcript shown on the confirm card
VA-04	Every write intent shows a confirm card (photo, name, village, amount in 48 sp digits, items) and speaks it; save only on confirm word or tap

ID	Rule
VA-05	If the amount is more than 3× the customer's median entry, ask a second confirmation
VA-06	Two failures in a row → open the manual form pre-filled with whatever was parsed
VA-07	Replies use ARB templates with placeholders; numbers formatted in Indian style (₹1,25,000)
VA-08	Replies are spoken in the language of the command (detected), else the app language
VA-09	Every command writes a `voice_logs` row (text only, never audio) with outcome
VA-10	Works offline on Android with the on-device recognizer; if no Kannada model is installed, prompt once to download it, else fall back to tapping
VA-11	Cancel words at any point discard the command

Reply templates (ARB keys, English shown)

Key	English	Placeholders
`vaBalance`	{name} owes {amount}. Last payment {date}.	name, amount, date
`vaConfirmCredit`	{amount} credit to {name} for {items}. Correct?	amount, name, items
`vaSavedCredit`	Saved. {name} now owes {balance}.	name, balance
`vaConfirmPayment`	{amount} received from {name} by {mode}. Correct?	amount, name, mode
`vaToday`	Today {sales} sales: cash {cash}, UPI {upi}, credit {credit}. Collected {collected}.	5 amounts
`vaStock`	{product}: {qty} {unit} left, about {days} days.	product, qty, unit, days
`vaNotUnderstood`	Sorry, I didn't understand. Please say it again or tap to type.	—

Key	English	Placeholders
`vaWhichCustomer`	Which {name}?	name

7. Functional requirements (SRS)

Each requirement has an acceptance check the AI MUST turn into a test. "Client" maps to the client requirements doc; "M" is the build step that delivers it. All are Must unless marked (S) Should or (C) Could.

7.1 Auth and setup (AU)

ID	Requirement	Acceptance check	Client	M
FR-AU-01	First launch shows a language picker (ಕನ್ನಡ, हिंदी, English) as 3 big buttons in their own script	Choosing one switches all UI text immediately; choice persists after restart	C-FR-02	M2
FR-AU-02	Sign in with Google or email OTP	New user lands on Create shop; returning user lands on PIN	C-FR-01	M2
FR-AU-03	Create shop: shop name, owner name, village, taluk, district, state (pick list), phone (optional), UPI ID (optional)	Calls API-03; shop + owner member exist; works for any district	C-FR-01	M2
FR-AU-04	Set a 4-digit PIN (enter twice); optional fingerprint	PIN hash stored; wrong PIN 5 times → 30 s lock, then sign-in again after 10	C-FR-03	M2
FR-AU-05	App locks after 2 minutes in background; unlock with PIN/fingerprint	Verified on device without network	C-FR-03	M2
FR-AU-06	Owner invites helper by email; helper signs in and accepts	Helper sees the shop with helper rights only	C-FR-04	M2

ID	Requirement	Acceptance check	Client	M
FR-AU-07	Import opening balances: list screen, one row per customer (name, village, amount)	50 rows entered in under 5 minutes; each creates a customer with `opening_balance_paise`	C-FR-05	M2
FR-AU-08 (S)	Starter product list: top 60 kirana items in kn/hi/en with units; owner ticks what they sell and sets prices	Ticked items created as products	C-FR-06	M2
FR-AU-09 (S)	Same account on web: sign in on browser, enter PIN, full data after sync	Web shows same balances as phone	C-FR-07	M4

7.2 Customers (CU) and khata (KH)

ID	Requirement	Acceptance check	Client	M
FR-CU-01	Add/edit customer: name (required), village, phone (10 digits, starts 6–9), photo, language, nickname, credit limit	Validation messages in app language; `name_latin` and `phonetic_key` computed on save	C-FR-10	M3
FR-CU-02	Customer list: photo, name, village, balance (red if owes, green if ⩽ 0); sort by name, balance, oldest unpaid	500 customers scroll at 60 fps on a 2 GB phone	C-FR-13	M3
FR-CU-03	Search by typing part of name/village/phone, or by voice	Results update per keystroke in < 100 ms locally	C-FR-18	M3
FR-KH-01	Add credit: amount (required), items text or product picks, date (default now), note	Row in `ledger_entries` with positive amount; balance updates instantly	C-FR-11	M3
FR-KH-02	Add payment: amount, mode cash/UPI, date; part payment allowed; payment larger than balance allowed with warning (advance)	Negative amount stored; balance updates	C-FR-12	M3

ID	Requirement	Acceptance check	Client	M
FR-KH-03	Customer khata screen: balance at top, entries newest first with date, items, +/− amount, running balance, who entered	Running balance matches `v_customer_balance`	C-FR-13	M3
FR-KH-04	Total outstanding (sum of positive balances) on Home	Matches SQL sum	C-FR-14	M3
FR-KH-05	Overdue list filtered by balance ≥ X and/or no payment for N days	Matches reference query	C-FR-15	M3
FR-KH-06	Undo own entry within 10 minutes (sets `deleted_at`, writes audit row)	After 10 minutes the Undo button disappears	C-FR-16	M3
FR-KH-07	Owner correction: creates `adjustment` with signed amount and required reason	Helper does not see the option; RLS rejects helper adjustments	C-FR-16	M3
FR-KH-08 (S)	Credit limit warning when a credit would cross the limit; owner may continue	Warning dialog shows limit and new balance	C-FR-17	M3
FR-KH-09	Helper entries above `helper_pin_limit_paise` need owner PIN on the same device	Without PIN, save is blocked	C-NFR-06	M3

7.3 Suppliers (SU)

ID	Requirement	Acceptance check	Client	M
FR-SU-01 (S)	Add supplier: name, phone, town, opening due	Supplier listed with due	C-FR-45	M3
FR-SU-02 (S)	Record purchase on credit (amount, optional bill photo, optional stock lines)	Positive `supplier_entries`; stock lines create `stock_movements` linked by `supplier_entry_id`	C-FR-45	M3
FR-SU-03 (S)	Record payment to supplier (amount, mode)	Due reduces	C-FR-45	M3

ID	Requirement	Acceptance check	Client	M
FR-SU-04 (S)	Supplier list with total due; supplier ledger like the customer khata	Matches `v_supplier_due`	C-FR-45	M3

7.4 Stock (ST)

ID	Requirement	Acceptance check	Client	M
FR-ST-01	Product: local name, Latin name, aliases, unit, sell price, cost price, reorder level, category	Saved; appears in billing grid	C-FR-40	M3
FR-ST-02	Billing reduces stock automatically (`sale` movements)	Stock on hand = opening + in − out	C-FR-41	M3
FR-ST-03	Add stock (purchase), damage/expiry write-off, manual count adjustment (owner, reason)	Each creates one movement row	C-FR-41	M3
FR-ST-04	Low-stock list: qty on hand ≤ reorder level, shown on Home as a badge	Count matches query	C-FR-42	M3
FR-ST-05 (S)	Days of stock left = qty ÷ average daily sales over last 30 days	Shown on product and in voice reply	C-FR-43	M3
FR-ST-06 (C)	Expiry date per stock-in; alert 7 days before	Alert listed on Home	C-FR-44	Later

7.5 Billing (BL)

ID	Requirement	Acceptance check	Client	M
FR-BL-01	Quick bill: tap product tiles (photo/icon + local name) or speak items; edit qty; free-text item allowed	Total = sum of lines	C-FR-30	M3

ID	Requirement	Acceptance check	Client	M
FR-BL-02	Payment split: cash / UPI / credit (any mix); credit part requires a customer	`sales` row satisfies the split CHECK; credit part also creates a `ledger_entries` credit linked by `sale_id`	C-FR-30	M3
FR-BL-03 (S)	UPI QR for the exact amount from `shops.upi_vpa`	QR encodes `upi://pay?pa=…&pn=…&am=…&cu=INR&tn=…`	C-FR-34	M6
FR-BL-04 (S)	Share bill as text/image on WhatsApp	Opens WhatsApp with bill summary	C-FR-23	M6
FR-BL-05 (C)	Bluetooth 58 mm printer receipt	Prints Kannada as image	C-FR-35	Later
FR-BL-06 (C)	GST invoice when `gst_enabled`	Behind feature flag	C-FR-36	Later

7.6 Daily summary and reports (DY)

ID	Requirement	Acceptance check	Client	M
FR-DY-01	Today: total sales, cash, UPI, credit given, old dues collected (cash/UPI), expected cash in drawer = cash sales + cash collections	Each figure matches SQL in section 12 tests	C-FR-31	M3
FR-DY-02 (S)	Week and month view with same figures + top 10 items	Correct across month boundaries in shop timezone	C-FR-32	M3
FR-DY-03 (S)	Rough profit = Σ(line total − qty × cost price) for the period; items without cost marked "unknown"	Matches reference query	C-FR-33	M3
FR-DY-04 (S)	Daily summary to the owner's own WhatsApp (share button; optional reminder at closing time)	Opens WhatsApp with summary text	C-FR-50	M6
FR-DY-05	Export full khata (all customers, all entries) as PDF and CSV	Files open; totals match app	C-FR-51	M7

ID	Requirement	Acceptance check	Client	M
FR-DY-06	Audit view: who entered/changed what and when (owner only)	Lists `audit_log` and entry creators	C-FR-52	M7

7.7 Notifications to customers (NT)

ID	Requirement	Acceptance check	Client	M
FR-NT-01	Send one customer a reminder by SMS (`sms:` intent) or WhatsApp (`wa.me` link), pre-filled in the customer's language	Message shows shop name and amount only; a `reminders` row is written	C-FR-20, 22	M6
FR-NT-02	Bulk: select overdue customers, then send one after another (app returns to the list after each)	Progress "3 of 7 sent"	C-FR-21	M6
FR-NT-03	Reminders only to customers with `reminder_opt_in = true`; opt-in captured when adding customer	Others greyed out with reason	C-FR-22	M6
FR-NT-04 (S)	Optional receipt after each credit/payment	Toggle in settings	C-FR-23	M6
FR-NT-05 (S)	Last reminded date shown on customer	From `reminders`	C-FR-24	M6
FR-NT-06 (C)	Voice-note reminder generated by TTS	Later	C-FR-25	Later

Reminder templates (exact text; `{shop}`, `{amount}`, `{upi_link}` placeholders)

Lang	Text
kn	ನಮಸ್ಕಾರ, {shop} ಅಂಗಡಿಯಲ್ಲಿ ನಿಮ್ಮ ಬಾಕಿ {amount}. ನಿಮಗೆ ಸಮಯ ಇದ್ದಾಗ ಪಾವತಿಸಿ. ಧನ್ಯವಾದಗಳು. {upi_link}

Lang	Text
hi	नमस्ते, {shop} में आपका बाकी {amount} है। समय मिले तो कृपया जमा करें। धन्यवाद। {upi_link}
en	Namaste, your balance at {shop} is {amount}. Please pay when convenient. Thank you. {upi_link}

The client must approve these texts before M6 (open item OI-03).

8. Wireframes and screen specs

Every screen follows one layout: top bar (title + sync dot), content, bottom navigation with 4 tabs (Home, Customers, Bill, Stock) and the mic button floating in the centre. Settings, suppliers and reports open from Home.

UI rules (apply to every screen)

ID	Rule
UI-01	Touch targets ≥ 48 dp; primary buttons ≥ 56 dp tall, full width
UI-02	Fonts: Noto Sans Kannada / Noto Sans Devanagari / Noto Sans; body 16 sp, amounts 20–48 sp bold
UI-03	Colours: primary ink `#1E2A3A`, background `#F7F4EC`, accent `#D9822B`; owes = red `#B3261E` with "ಬಾಕಿ/बाकी/Due" label; paid = green `#1B7F3B` with label (never colour alone)
UI-04	Every icon has a text label; max 2 words per button
UI-05	Amounts always ₹ with Indian grouping (₹1,25,000); dates as "3 Nov" in the app language
UI-06	Every list has an empty state with one action ("Add your first customer")
UI-07	Every save shows a snackbar with Undo for 10 s (and the 10-minute undo stays on the entry)
UI-08	No screen needs typing for daily work: voice or tap pickers first, keyboard second

ID	Rule
UI-09	Works at 360 × 640 dp; web layout max width 1200 px with a left rail instead of bottom tabs

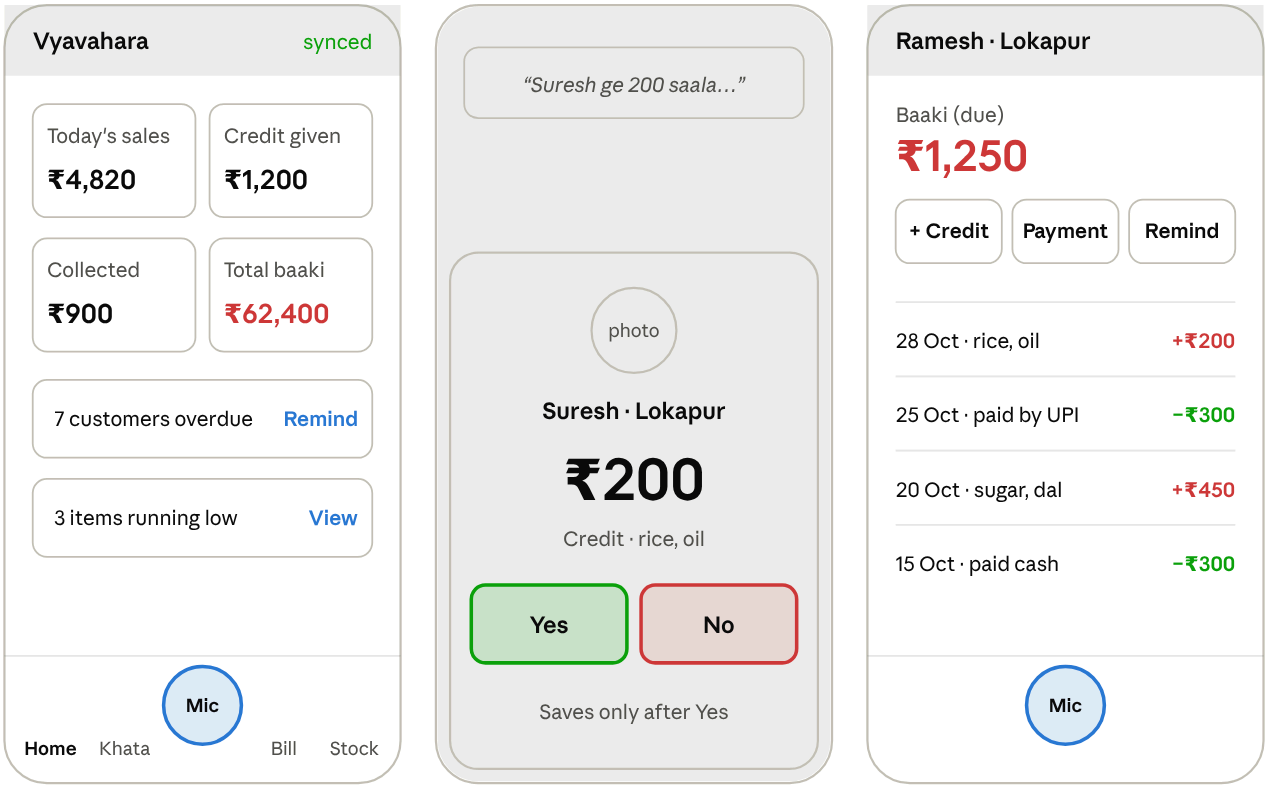
Screen list

ID	Screen	Route	Main elements	Actions	M
SCR-01	Language picker	`/lang`	3 big buttons in own script	Pick → SCR-02	M2
SCR-02	Sign in	`/login`	Logo, tagline, "Continue with Google", "Email code"	Sign in → SCR-03 or SCR-05	M2
SCR-03	Create shop	`/setup/shop`	Shop name, owner, village, taluk, district, state pickers, UPI ID	Save → SCR-04	M2
SCR-04	Set PIN	`/setup/pin`	4-dot PIN pad twice, fingerprint toggle	Done → SCR-06 (import)	M2
SCR-05	Unlock	`/unlock`	Shop name, PIN pad, fingerprint button	Correct PIN → last screen	M2
SCR-06	Import old khata	`/setup/import`	Rows: name, village, amount; "+ row"; voice add	Finish → SCR-07 (starter items)	M2
SCR-07	Starter items	`/setup/items`	Grid of 60 items with tick + price field	Finish → SCR-08	M2
SCR-08	Home	`/`	4 tiles: Today's sales, Credit given today, Collected today, Total baaki; cards: overdue count, low-stock count, sync status; shortcuts: Suppliers, Reports, Reminders, Settings	Tap tile → detail; mic	M3
SCR-09	Voice listening sheet	overlay	Pulsing mic, live transcript, language chip, Cancel	Auto-stop on silence 1.5 s	M5

ID	Screen	Route	Main elements	Actions	M
SCR-10	Voice confirm card	overlay	Customer photo, name, village; amount in 48 sp; items; green "Yes" and red "No" buttons; disambiguation list when needed	Yes → save + reply; No → edit form	M5
SCR-11	Customer list	`/customers`	Search bar + mic, sort chips, rows (photo, name, village, balance, last payment)	Tap → SCR-12; FAB add	M3
SCR-12	Customer khata	`/customers/:id`	Header: photo, name, phone, balance big; buttons "+ Credit", "Got payment", "Remind", "Call"; entries list with running balance	Entry long-press → undo/correct	M3
SCR-13	Add credit / payment form	`/customers/:id/entry`	Amount keypad (big), items picker, date, mode (payment), note	Save	M3
SCR-14	Quick bill	`/bill`	Product tile grid with search, cart strip with total, Cash / UPI / Credit buttons, split option	Pay → receipt / share	M3
SCR-15	Stock	`/stock`	Product rows with qty bar, low-stock first; "Add stock"	Tap → product detail	M3
SCR-16	Suppliers	`/suppliers`	Supplier rows with due; supplier ledger	Purchase / payment	M3
SCR-17	Day summary & reports	`/reports`	Today / Week / Month tabs; figures of FR-DY-01; top items; profit (owner)	Share to WhatsApp, export	M3
SCR-18	Reminders	`/reminders`	Overdue list with tick boxes, filters (≥ ₹X, ≥ N days), channel toggle	Send one by one	M6
SCR-19	Settings	`/settings`	Shop details, UPI ID, language, helpers, PIN, helper limit, sync problems, export, about	—	M2–M7

Drawn wireframes of the three most-used screens follow at the end of this section.

Three screens carry most of a merchant day



SCR-08 Home | **SCR-10 Voice confirm** | **SCR-12 Customer khata**

wireframes · SCR-08, SCR-10, SCR-12

Low-fidelity layout only: positions, hierarchy and labels are binding; exact colours and spacing come from UI-01 to UI-09. Red amounts are dues, green amounts are payments, and each carries a text label as well.

9. Non-functional requirements

Every target below is measurable and MUST have an automated or scripted check before M7 sign-off.

ID	Area	Requirement	How it is checked
NFR-01	Offline	All daily work (credit, payment, bill, stock, balance, summary, voice on Android) works in airplane mode	Integration test with network disabled

ID	Area	Requirement	How it is checked
NFR-02	Devices	Smooth on Android 9+, 2 GB RAM (reference: any ₹7,000–8,000 phone)	Manual test on 2 real low-end phones each milestone
NFR-03	App size	Release APK ⩽ 30 MB (split per ABI)	CI fails above limit
NFR-04	Start-up	Cold start to Home ⩽ 3 s on reference phone	Measured in profile build
NFR-05	Speed	Local save ⩽ 200 ms; list of 500 customers scrolls at 60 fps; search ⩽ 100 ms	Benchmarks in `integration_test`
NFR-06	Voice latency	Final transcript to confirm card ⩽ 1 s; spoken reply starts ⩽ 2 s	Logged timings
NFR-07	Voice accuracy	⩾ 85% of the 60-phrase test set per language parsed right first time; 100% correct after confirm	`voice_accuracy_test.dart` + field test AC-01
NFR-08	Data safety	No entry lost or duplicated across offline periods up to 30 days	Sync soak test with 2 devices
NFR-09	Recovery	New device restores 1 year of a 500-customer shop in ⩽ 5 minutes on 4G	Bootstrap test
NFR-10	Data use	⩽ 5 MB mobile data per shop per day (photos excluded)	Network logging
NFR-11	Battery	No background work except sync every 15 min max via WorkManager; no wake locks	Code review
NFR-12	Languages	100% of UI strings in kn, hi, en; no hard-coded text	CI script checks ARB key parity

ID	Area	Requirement	How it is checked
NFR-13	Accessibility	Contrast ⩾ 4.5:1; every icon labelled; screen reader labels on buttons	Flutter accessibility guideline tests
NFR-14	Availability	App never blocks on the server; cloud target 99%	Design rule; Supabase status
NFR-15	Capacity	500 customers, 300 products, 50,000 ledger rows per shop without slowdown	Seeded performance test
NFR-16	Maintainability	`flutter analyze` with zero warnings; test coverage ⩾ 70% for `domain`, `data`, `voice`	CI
NFR-17	Cost	Stays inside Supabase and GitHub free limits at pilot scale	Nightly job reports DB size

10. Security, errors and edge cases

Security rules

ID	Rule
SEC-01	Row Level Security on every table; every policy keyed on `shop_id` membership (section 4)
SEC-02	Edge Functions use the caller's JWT, never the service-role key, for data access
SEC-03	Local SQLite encrypted with SQLCipher on Android; key generated on first run and kept in Android Keystore via `flutter_secure_storage`
SEC-04	PIN stored only as an argon2id hash; never logged or synced in clear

ID	Rule
SEC-05	No secrets in the repo: `.env` git-ignored; CI uses GitHub secrets; a secret-scanning step runs on every PR
SEC-06	Customer data limited to name, village, phone, photo, language; no Aadhaar, no bank details
SEC-07	Logs and Crashlytics MUST NOT contain names, phones or amounts
SEC-08	Voice audio is never stored or uploaded by the app
SEC-09	Storage bucket `shop-media` private; signed URLs valid 1 hour
SEC-10	Owner can export and request deletion of all shop data; deletion after export confirmation (DPDP Act)

Error handling

Situation	App behaviour
No network	Everything continues; sync dot turns orange; no error popups
Session expired	Keep working offline; ask to sign in again only when sync needs it; never lose outbox rows
Sync row rejected (`RLS_DENIED` , `VALIDATION` , `IMMUTABLE`)	Row stays local, marked with a warning icon; listed in Settings › Sync problems with plain-language reason
`FK_MISSING`	Retry next sync after parents push
Speech engine missing language	One-time prompt to download offline Kannada/Hindi pack; else hide mic and show tap entry
Storage full on phone	Warn at 100 MB free; block photo capture, never block entries

Situation	App behaviour
App crash during save	Transaction rolls back; nothing half-saved

Edge cases the code MUST handle

1. Two customers with the same name → disambiguation (section 6).
2. Payment larger than balance → allowed, balance goes negative (advance), shown in green as "advance".
3. Same customer edited on two phones offline → last `updated_at` wins; the other version goes to `audit_log`.
4. Helper undoes an entry offline after 9 minutes, syncs after 2 hours → accepted (rule uses `deleted_at - created_at`).
5. Phone clock wrong → server stores its own `server_seq`; display uses `entry_time` but warns if device time differs from server by > 10 minutes at sync.
6. Bill with credit part but no customer → blocked by validation and by the database CHECK.
7. Product sold below zero stock → allowed (village reality), stock shows negative in red with "check stock".
8. Customer deleted (soft) with non-zero balance → owner must confirm; entries stay for history.
9. Month and day boundaries computed in `shops.timezone`, not UTC.

11. Build steps M0–M7

Build in this order; each step ends with a reviewed pull request that passes CI and its definition of done. Never start a step before the previous one is merged. About 12 weeks for a 2–3 person team.

Standard preamble — paste before every step prompt

```
You are building "Vyavahara" (tagline: ನಿಮ್ಮ ಪೂರ್ತಿ ವ್ಯವಹಾರ, ಒಂದೇ ಕಡೆ), an offline-
first
Flutter (Android + web) app on Supabase for village merchants. The file
docs/srs.md in this repo is the single source of truth. Follow it literally.
Rules:
- Implement ONLY the build step named below. Read the sections it lists
first.
- Use the fixed tech stack and repo layout in section 3. Do not add packages
  not listed there without asking.
- Use exact table/column names from section 4 and IDs from sections 5-8.
- Write tests for every acceptance check you implement.
- All user-facing text goes in ARB files (kn, hi, en). No hard-coded strings.
- Work on branch feat/m<N>-<name>; Conventional Commits; open a PR listing
the
  IDs implemented; never push to main; never commit secrets.
- If anything is unclear or missing in docs/srs.md, STOP and ask.
Start by posting a short plan (files, tests, open questions) and wait for
"go".
```

M0 — Repository and CI (week 1)

- Read first: sections 0, 3.
- Deliver: monorepo layout; Flutter app with Riverpod, go_router, theme (UI-02, UI-03), l10n with 3 ARB files and a language switch; Supabase local setup (`supabase/config.toml`); CI workflow (`flutter analyze`, `flutter test`, build APK + web, ARB key-parity script, APK size check ⩽ 30 MB, secret scan); README, CONTRIBUTING, PR and issue templates; `docs/srs.md` = this document.
- Done when: CI green on a PR; app shows a placeholder Home in kn/hi/en on Android and web.
- Prompt: `Build step M0 — Repository and CI. Deliver everything listed under M0 in section 11.`

M1 — Data layer (weeks 2–3)

- Read first: sections 4, 5 (device tables), 10.
- Deliver: migrations `0001_init.sql` and `0002_rls.sql` (all tables, triggers, views, full RLS for every table and the immutability trigger); drift schema mirroring Postgres + `outbox` + `sync_state`; DAOs; repositories per feature that write row + outbox in one transaction; domain models with `Money` and `Qty` value objects; seed data for dev.

- Done when: SQL tests prove RLS (member can read/insert, helper cannot adjust, nobody can hard-delete, immutable ledgers); Dart tests prove balance, supplier due and stock queries match the Postgres views on the same seed data.
- Prompt: `Build step M1 — Data layer. Implement section 4 exactly, both in supabase/migrations and in drift.`

M2 — Auth and onboarding (week 4)

- Read first: sections 2, 7.1, 8 (SCR-01 to SCR-07), API-03 to API-05, SEC-03, SEC-04.
- Deliver: SCR-01 to SCR-07; Google + email OTP sign-in; RPCs `create_shop`, `invite_helper`, `accept_invite`; PIN set/unlock with lockout; app lock after 2 min; import old khata; starter item list (60 items in 3 languages as a JSON asset).
- Done when: FR-AU-01 to FR-AU-08 acceptance checks pass; a helper can join a shop and has limited rights.
- Prompt: `Build step M2 — Auth and onboarding (FR-AU-01..08).`

M3 — Core screens, fully offline (weeks 5–7)

- Read first: sections 7.2–7.6, 8 (SCR-08, SCR-11 to SCR-17), 10 (edge cases).
- Deliver: Home tiles; customers list and khata; credit/payment forms with big keypad; undo and owner adjustment; helper PIN limit; suppliers; products and stock; quick bill with payment split; day/week/month summary and profit.
- Done when: all FR-CU, FR-KH, FR-SU, FR-ST, FR-BL-01/02, FR-DY-01..03 checks pass in airplane mode; NFR-05 benchmarks pass with 500 seeded customers.
- Prompt: `Build step M3 — Core screens offline. Implement the FR IDs listed under M3, using the repositories from M1. No network code in this step.`

M4 — Sync engine (weeks 8–9)

- Read first: section 5 entirely, 10 (errors, edge cases 3–5).
- Deliver: Edge Functions `sync-push` and `sync-pull` (TypeScript, with tests); `core/sync/sync_engine.dart` with triggers, mutex, backoff, cursors, bootstrap screen, sync dot, Settings › Sync problems; web build working against the same account (FR-AU-09).
- Done when: two-device soak test (NFR-08) passes: 2 phones offline 1 day, 200 entries each, reconnect → identical balances on both and on web, no duplicates; NFR-09 restore test passes.
- Prompt: `Build step M4 — Sync engine. Implement section 5 exactly, API-01 and API-02 contracts included.`

M5 — Voice assistant (weeks 10–11)

- Read first: section 6 entirely, SCR-09, SCR-10, NFR-06, NFR-07.
- Deliver: `app/lib/voice/` pipeline (STT adapters, normaliser, transliteration table, number parser, intent grammar for all intents in kn/hi/en, resolver with disambiguation and memory, reply builder); listening sheet and confirm card; `voice_logs`; `docs/voice-grammar.md` with the full lexicon; test sets of ≥ 60 phrases per language.
- Done when: NFR-07 accuracy test ≥ 85% per language; every write intent requires confirmation; VA-01 to VA-11 checks pass; works offline on Android.
- Prompt: `Build step M5 — Voice assistant. Implement section 6. The pipeline must be pure Dart except the STT/TTS adapters, and must never write data without the confirm card.`

M6 — Reminders, UPI and sharing (week 12, first half)

- Read first: section 7.7, FR-BL-03/04, FR-DY-04, SCR-18.
- Deliver: single and bulk reminders via SMS intent and WhatsApp link with approved templates; opt-in handling; reminder log; UPI QR and link; share bill and day summary to WhatsApp.
- Done when: FR-NT-01..05, FR-BL-03/04, FR-DY-04 checks pass on a real phone.
- Prompt: `Build step M6 — Reminders and UPI (FR-NT-01..05, FR-BL-03, FR-BL-04, FR-DY-04).`

M7 — Hardening and release (week 12, second half + pilot)

- Read first: sections 9, 10, 12.
- Deliver: SQLCipher; export PDF/CSV (FR-DY-05); audit view (FR-DY-06); Crashlytics without personal data; performance pass on a 2 GB phone; signed release APK; web deploy to GitHub Pages or Cloudflare Pages; privacy policy and terms pages; Play Store listing text in 3 languages; nightly job API-07.
- Done when: every NFR check passes; acceptance tests AC-01 to AC-10 from the client doc pass in 3 pilot shops.
- Prompt: `Build step M7 — Hardening and release. Implement the deliverables listed under M7 and run every NFR check in section 9.`

Later (do not build in version 1): GST billing, Bluetooth printer, expiry alerts, voice-note reminders, Bhashini/Sarvam STT, platform-fee billing, Power BI admin view.

12. Testing strategy and traceability

Every FR, VA and NFR ID maps to at least one automated test whose name contains the ID, so a search for `FR-KH-03` finds its test.

Level	Tool	What it covers	Where	Runs
Unit	`flutter_test`, `mocktail`	Money/Qty, number parser, intent grammar, resolver, balance maths, reply templates	`app/test/`	Every PR
Database	SQL tests (pgTAP or plain SQL asserts via `supabase test db`)	RLS allow/deny matrix, CHECK constraints, immutability trigger, views	`supabase/tests/`	Every PR
Edge Functions	Deno test	sync-push/pull contracts, rejection codes, conflict rule	`supabase/functions/*/test.ts`	Every PR
Widget	`flutter_test`	Each screen's empty, loading, filled, error states	`app/test/ui/`	Every PR
Integration	`integration_test` on emulator	End-to-end flows A–G below, airplane mode	`app/integration_test/`	Nightly + before merge of M3+
Field	Real phones in 3 shops	Client acceptance tests AC-01 to AC-10	Pilot	M7

End-to-end flows to automate

1. A — Sign up, create shop, set PIN, import 5 customers, add 5 items.
2. B — Credit sale by voice (simulated transcript) with disambiguation and confirm.
3. C — Payment received, part payment, advance payment.
4. D — Quick bill with cash + credit split; stock reduces; ledger credit created.

5. E — Day summary figures match seeded expectations.
6. F — Two devices offline, reconcile after sync.
7. G — Bulk reminders open the right intents with the right text.

Reference SQL for report tests (Postgres; mirror in drift)

```
-- Today summary for a shop (in shop timezone)
WITH b AS (
  SELECT (date_trunc('day', now() AT TIME ZONE s.timezone) AT TIME ZONE
s.timezone) AS day_start
  FROM shops s WHERE s.id = :shop_id
)
SELECT
  COALESCE(SUM(sa.total_paise), 0)  AS sales_paise,
  COALESCE(SUM(sa.cash_paise), 0)   AS cash_paise,
  COALESCE(SUM(sa.upi_paise), 0)    AS upi_paise,
  COALESCE(SUM(sa.credit_paise), 0) AS credit_from_bills_paise,
  (SELECT COALESCE(SUM(amount_paise), 0) FROM ledger_entries l, b
    WHERE l.shop_id = :shop_id AND l.entry_type = 'credit' AND l.deleted_at
IS NULL
      AND l.entry_time >= b.day_start AND l.entry_time < b.day_start +
interval '1 day') AS credit_given_paise,
  (SELECT COALESCE(-SUM(amount_paise), 0) FROM ledger_entries l, b
    WHERE l.shop_id = :shop_id AND l.entry_type = 'payment' AND l.deleted_at
IS NULL
      AND l.entry_time >= b.day_start AND l.entry_time < b.day_start +
interval '1 day') AS collected_paise,
  (SELECT COALESCE(-SUM(amount_paise), 0) FROM ledger_entries l, b
    WHERE l.shop_id = :shop_id AND l.entry_type = 'payment' AND
l.payment_mode = 'cash' AND l.deleted_at IS NULL
      AND l.entry_time >= b.day_start AND l.entry_time < b.day_start +
interval '1 day') AS collected_cash_paise
FROM b
LEFT JOIN sales sa ON sa.shop_id = :shop_id AND sa.deleted_at IS NULL
  AND sa.sale_time >= b.day_start AND sa.sale_time < b.day_start + interval
'1 day';
-- expected cash in drawer = cash_paise + collected_cash_paise

-- Overdue list
SELECT customer_id, full_name, village, balance_paise, last_payment_at
FROM v_customer_balance
WHERE shop_id = :shop_id AND balance_paise >= :min_paise
  AND COALESCE(last_payment_at, last_credit_at) < now() - make_interval(days
```

```
=> :days)
ORDER BY balance_paise DESC;

-- Low stock with days left
SELECT s.product_id, s.name_local, s.qty_on_hand, s.reorder_level,
       ROUND(s.qty_on_hand / NULLIF(-SUM(m.qty_change) / 30.0, 0), 1) AS
days_left
FROM v_stock_on_hand s
LEFT JOIN stock_movements m ON m.product_id = s.product_id AND
m.movement_type = 'sale'
  AND m.deleted_at IS NULL AND m.movement_time >= now() - interval '30 days'
WHERE s.shop_id = :shop_id AND s.qty_on_hand <= s.reorder_level
GROUP BY s.product_id, s.name_local, s.qty_on_hand, s.reorder_level
ORDER BY days_left NULLS LAST;
```

Open items (must be closed before the step that needs them)

ID	Question	Needed by	Owner
OI-01	Recorded voice samples from 3–5 merchants (with consent)	M5	Client
OI-02	2–3 photos of real paper khata pages	M2	Client
OI-03	Approval of reminder texts in kn/hi/en	M6	Client
OI-04	Can two phones in one shop work at the same time? (design assumes yes)	M4	Client
OI-05	Platform fee amount and start date (affects `plan` field only)	Later	Client
OI-06	Data retention after a merchant leaves	M7	Client + vendor
OI-07	App icon and logo for Vyavahara	M7	Vendor